SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-019: Student Profile Editing

Field	Details
Change Request ID	CR-019
Change Request Title	Student Profile Editing
Student Name	C. Lavasujan
Student ID	24ICT06
Implementation Status	Implemented; end-to-end verification pending
Date	29 October 2026

1. Change Request Summary

1.1 Business Requirement

CR-019 allows an authenticated student to update their own email address and phone number directly from the Student Dashboard. The feature is intended to keep contact details current without requiring administrator assistance, while preserving the existing profile display, authentication and administrator operations.

1.2 Existing Behaviour

Before this change, the My Profile section displayed registration number, full name, email, phone number and member-since date as read-only information. Students had no profile-editing form. The existing PUT /api/students/:id route was intended for administrator updates, while student profile retrieval used GET /api/students/me.

1.3 Requested Change and Scope

- Add an Edit Profile action and pre-populated form for the student's email address and phone number.
- Validate email and phone on both client and server; reject malformed or missing values with a clear response.
- Add a protected PUT /api/students/me endpoint that identifies the student from the verified JWT user context.
- Update only the authenticated student record, handle duplicate email as a conflict, and refresh the displayed profile after saving.
- Keep registration number, full name and member-since read-only; do not change passwords, usernames, database schema, or administrator CRUD behaviour.

2. Implementation

2.1 Implementation Summary

The supplied implementation notes describe changes across the student routes, controller, model, StudentDashboardPage.jsx, and studentDashboard.css. A protected PUT /api/students/me endpoint uses JWT authentication and the STUDENT role. The backend derives the target student from req.user.userId, validates contact fields, checks duplicate email, and updates only email and phone_number in the existing record. The dashboard provides edit, save and cancel actions with success/error feedback.

2.2 Affected Files

File	Change	Purpose
`backend/src/routes/studentRoutes.js`	Modified	Registered protected student-only PUT /me route.
`backend/src/controllers/studentController.js`	Modified	Validates input, resolves the authenticated profile, handles duplicate-email conflicts and formats responses.
`backend/src/models/studentModel.js`	Modified	Adds email uniqueness lookup and contact-only update scoped by user ID.
`frontend/src/pages/StudentDashboardPage.jsx`	Modified	Adds edit form, populated contact fields, save/cancel handling and user feedback.
`frontend/src/styles/studentDashboard.css`	Modified	Styles contact inputs, buttons, feedback and narrow-screen layout.
CHANGELOG.md	Modified	Records CR-019 in the unreleased changes.

2.3 Technical Changes

API and authorization: PUT /api/students/me uses the existing JWT authenticate middleware and permits the STUDENT role. The endpoint derives the user ID from req.user.userId; it does not accept a student ID in the request body or URL. A successful update returns HTTP 200 and the updated record in the student property.

Validation and uniqueness: Email is trimmed, must have a basic valid structure and is limited to 150 characters. Phone is trimmed, accepts common formatting characters, is limited to 20 characters and must contain 7–15 digits. The backend checks duplicate email and translates duplicate-key conflicts to HTTP 409.

Data access: The model updates only email and phone_number for the authenticated user ID. Existing students.email and students.phone_number fields and the unique email constraint are reused; no schema migration or new environment variables/packages are required.

Dashboard behaviour: Edit contact details reveals required inputs prefilled with saved values. Save disables controls while submitting. On success, displayed data is replaced with the API response; on failure, the API error is shown. Cancel leaves saved data unchanged. Registration number, full name and member-since remain display-only.

2.4 Implementation Flow

1. The student signs in and opens the Student Dashboard; GET /api/students/me loads the current profile.
2. The student selects Edit contact details; current email and phone values populate the form.
3. The student edits one or both contact values and selects Save. The frontend checks input and sends the values to PUT /api/students/me with the authentication context.
4. The backend verifies the JWT and STUDENT role, resolves the student by req.user.userId, validates the values and checks email uniqueness.
5. The backend updates only email and phone_number. On HTTP 200, the dashboard refreshes the displayed profile and shows confirmation; errors are shown without updating the displayed profile.

2.5 Error and Data Handling

HTTP Status	Condition / Handling
400	Email or phone missing/invalid; invalid values are rejected before database update.
401	JWT is missing, invalid or expired; existing authentication handling applies.
403	Authenticated user does not have the STUDENT role.
404	No student profile is associated with the authenticated user.
409	Requested email is already registered to another student.
500	Unexpected failure prevents the profile update.

3. Testing & Verification

3.1 Checks Reported as Run

The supplied implementation report records successful build and targeted static checks. It explicitly notes that automated backend/API integration tests and manual end-to-end testing against a running database were not performed.

ID	Check	Recorded Result
CHK-01	Frontend production build: npm.cmd run build	Passed
CHK-02	ESLint on frontend/src/pages/StudentDashboardPage.jsx	Passed
CHK-03	node --check on the changed backend JavaScript files	Passed
CHK-04	git diff --check	Passed
CHK-05	Full frontend lint: existing diagnostics remain in untouched StudentFormModal.jsx, DashboardPage.jsx and ProtectedRoute.jsx	Not clean (pre-existing issues)
CHK-06	Automated backend/API integration tests	Not run; no automated test suite is defined in the backend package
CHK-07	Manual end-to-end testing with a running database	Not run

3.2 Recommended Manual Test Cases

Test ID	Manual Scenario / Expected Verification	Result
TC-01	Edit email and phone with valid values; save and verify the dashboard displays them.	Not run
TC-02	Change email only, then phone only; verify each saved value persists.	Not run
TC-03	Submit malformed email or phone; verify rejection and no stored change.	Not run
TC-04	Use another student’s email; expect HTTP 409 and no record changes.	Not run
TC-05	Cancel edit; verify saved values remain unchanged and no update occurs.	Not run
TC-06	Submit without a token and as an ADMIN; verify endpoint rejects both requests.	Not run
TC-07	Confirm registration number, full name and member-since cannot be edited.	Not run
TC-08	Regression-check GET /api/students/me, student login and administrator CRUD.	Not run

Verification note: “Passed” applies only to the checks listed as run above. End-to-end correctness, database persistence and authorization behaviour should be confirmed with the recommended manual cases before release.

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	Student can open Edit Profile from My Profile.	Dashboard edit action	Implemented; manual test pending
AC-002	Current email and phone values are pre-filled.	Edit form integration	Implemented; manual test pending
AC-003	Valid email can be saved.	PUT /api/students/me	Implemented; API test pending
AC-004	Valid phone can be saved.	Server-side phone validation	Implemented; API test pending
AC-005	Invalid email/phone is rejected with a clear message.	Frontend/backend validation	Implemented; negative test pending
AC-006	Updated values persist in the existing database record.	Model update by user ID	Implemented in code; DB test pending
AC-007	Updated values display after save.	Profile refreshed from API response	Implemented; manual test pending
AC-008	Student cannot update another student profile.	JWT-derived identity / route scope	Implemented; authorization test pending
AC-009	Existing student login and profile retrieval work.	Regression test	Not run end-to-end
AC-010	Existing administrator CRUD behaviour is unchanged.	Regression test	Routes noted unchanged; regression test pending

5. Evidence

Replace the placeholders below with screenshots captured from the actual application or test environment. Use test accounts and avoid exposing tokens or unnecessary personal data.

Evidence 01 - Student Dashboard Edit Profile Form

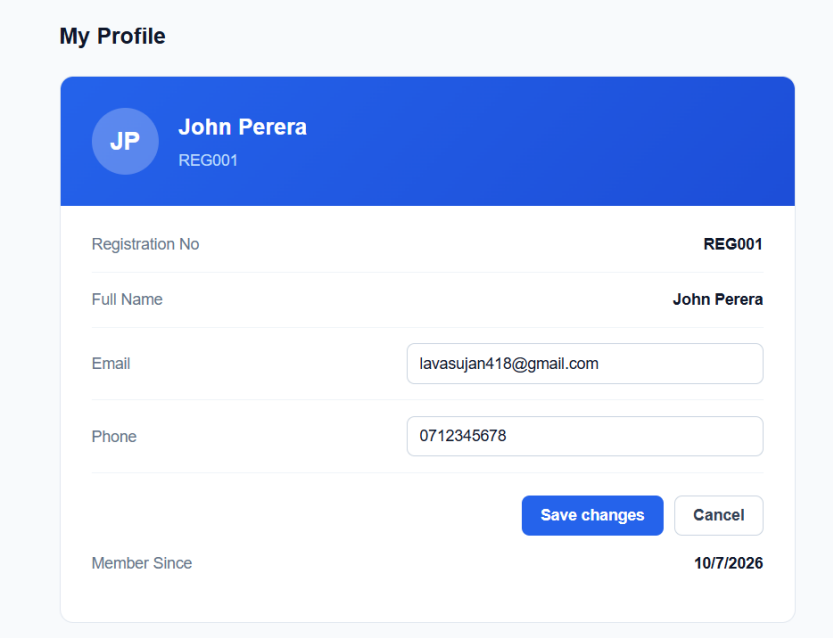


Figure 1: Student Dashboard showing the profile edit form with current contact values prefilled.

Evidence 02 - Successful Save and Validation Feedback

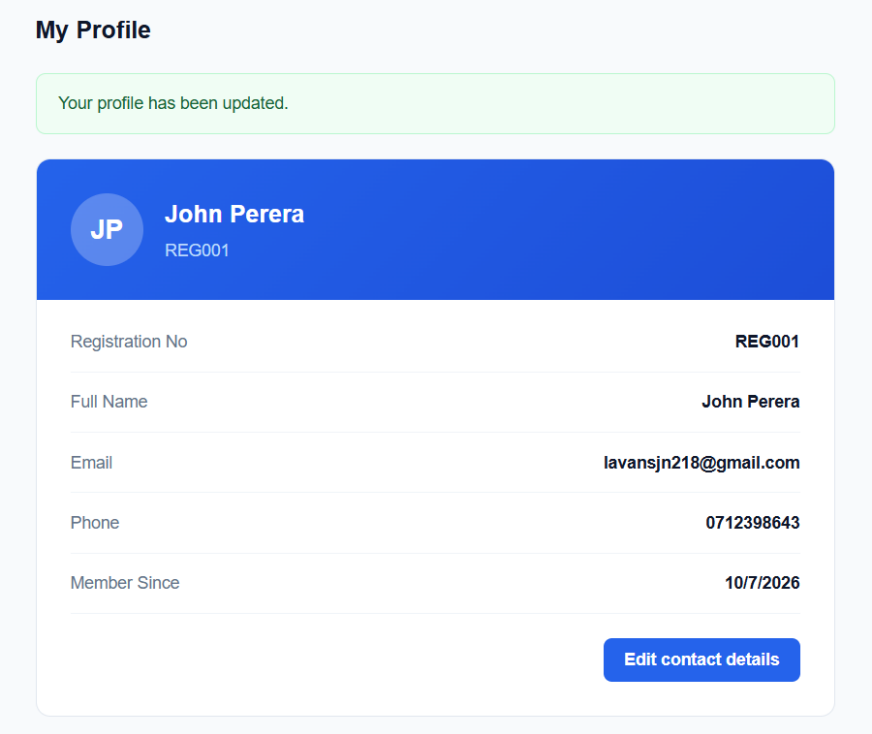


Figure 2: Updated Student Dashboard profile after a successful contact-detail update.

6. Final Result

Item	Result
Requirements	Implemented in supplied change description
Implementation	Reported as implemented
Build and targeted static checks	Passed
Full frontend lint	Not clean due to existing diagnostics in untouched files
`API integration / manual end-to-end tests`	Not run
Acceptance criteria	Code implementation documented; end-to-end verification pending
Database migration	Not required
Final Status	Implemented; verify manually before release

6.1 Final Summary

CR-019 adds self-service editing for a student's email address and phone number using the authenticated student context. The feature reuses the existing database structure, protects the update route with JWT authentication and the STUDENT role, validates contact fields, handles duplicate email conflicts, and keeps the other profile fields read-only. The frontend production build and targeted static checks passed according to the supplied implementation notes; complete API/database and manual end-to-end verification remain outstanding.

6.2 Issues / Limitations

- The full frontend lint is not clean because of reported diagnostics in untouched StudentFormModal.jsx, DashboardPage.jsx and ProtectedRoute.jsx.
- No automated backend/API integration suite was run, and manual testing against a running database was not performed.
- Before release, execute the manual test cases in Section 3.2, especially duplicate-email conflict, unauthenticated access, role restriction and regression checks.

Student Name: C. Lavasujan

Student ID: 24ICT06

Date: 29 October 2026

Signature: _____

~ END OF REPORT ~